High-Level Design

Legal Document RAG Assistant

Done By: VADIJARLA VINESH

Internship Project High-Level Design (HLD) Report

IN226077102

Contents

1 System Overview **2**
- 1.1 Problem Definition . . . 2
- 1.2 Objective . . . 2
- 1.3 Scope . . . 2

2 Architecture Diagram **2**

3 Component Description **3**
- 3.1 Document Loader . . . 3
- 3.2 Chunking Strategy . . . 3
- 3.3 Embedding Model . . . 3
- 3.4 Vector Store . . . 3
- 3.5 Retriever . . . 3
- 3.6 LLM . . . 3
- 3.7 Graph Workflow Engine . . . 3
- 3.8 Routing Layer . . . 3
- 3.9 HITL Module . . . 3

4 Data Flow **4**
- 4.1 Document Processing Flow . . . 4
- 4.2 Query Flow . . . 4

5 Technology Choices **4**
- 5.1 ChromaDB . . . 4
- 5.2 LangGraph . . . 4
- 5.3 Groq . . . 4

6 Scalability Considerations **4**
- 6.1 Handling Large Documents . . . 4
- 6.2 Increasing Query Load . . . 4
- 6.3 Latency Optimization . . . 5

1 System Overview

1.1 Problem Definition

Legal documents such as contracts and agreements are complex, lengthy, and difficult to interpret for non-experts. Users often need quick answers to specific queries without reading the entire document.

1.2 Objective

- Processes legal PDF documents
- Retrieves relevant information using embeddings
- Generates accurate, context-based answers
- Escalates complex queries to a human when required

1.3 Scope

- **Input:** Legal PDF documents
- **Output:** Contextual answers to user queries
- **Interface:** CLI-based system
- Includes Human-in-the-Loop (HITL) escalation

2 Architecture Diagram

User Query

↓

Intent Routing Layer

↓

Retriever (ChromaDB)

↓

LLM Processing (Groq API)

↓

Decision Layer

↓

Answer OR HITL Escalation

3 Component Description

3.1 Document Loader

Loads PDF documents using PyPDFLoader.

3.2 Chunking Strategy

- RecursiveCharacterTextSplitter
- Chunk size: 500
- Overlap: 100

3.3 Embedding Model

- sentence-transformers (all-MiniLM-L6-v2)
- Converts text into vector embeddings

3.4 Vector Store

ChromaDB used for storing embeddings locally.

3.5 Retriever

Retrieves top-k (k=3) relevant chunks.

3.6 LLM

- llama-3.3-70b-versatile (Groq API)
- Generates answers using retrieved context

3.7 Graph Workflow Engine

Built using LangGraph to control workflow execution.

3.8 Routing Layer

- Decides response path
- Routes to HITL if needed

3.9 HITL Module

Handles escalation and returns human-generated responses.

4 Data Flow

4.1 Document Processing Flow

PDF → Load → Chunk → Embed → Store

4.2 Query Flow

User Query → Routing → Retrieval → LLM → Response

5 Technology Choices

5.1 ChromaDB

- Lightweight
- Easy integration
- Local storage

5.2 LangGraph

- Workflow-based execution
- Conditional routing

5.3 Groq

- Fast inference
- High-quality reasoning

6 Scalability Considerations

6.1 Handling Large Documents

Chunking ensures efficient processing.

6.2 Increasing Query Load

Handled via API-based LLM scaling.

6.3 Latency Optimization

- Top-k retrieval
- Efficient embeddings